

dataEVT

REFERENCE

API consumer

interpret the reader's tag-read (and barcode) output

Overview

`dataEVT` is the reader's **tag-data** event — the device publishes it **autonomously** while an inventory is running, carrying the RFID tag reads (and any barcode reads) it produces. It is **device-emitted** (no command is in the payload path), but it only flows once you **start an inventory** with `control_operation` `START`, using whatever the `set_operating_mode` profile configured. It is read-only tag output.

Each event is a small envelope — `type` (the active profile), `timestamp`, and a `data` object holding a `tagData[]` array (and/or a `barcodeData[]` array). Reads are **batched** — the contract declares no limit on records per event — and **the operating mode's** `tagMetaDataToEnable` **flags gate which fields each tag carries** — of a record's twenty declared fields, only two are always there. See [Field Support](#).

`dataEVT` is published on the **DATA endpoint's** data topic (e.g. `<tenantId>/DATA/clients/data1event/<serial>` for `DATA1`), not the `event` topic (`alerts` / `heartBeatEVT`) or the `rfid` topic (`mqttConnEVT`). Receiving it requires an **active** `DATA1` **endpoint** (`config_endpoint`, `epType: DATA1`, `activate: true`). The contract also declares a second data channel, `DATA2`, but it is currently not supported.

Details

Property	Value
Event Type	Tag-data (and barcode) read output
Trigger	Inventory (started with <code>control_operation</code> <code>START</code> ; stops on <code>STOP</code>)
Applies To	Endpoint: Data (DATA). Reader: RFD40 series, RFD90 series
Credentials	None device-side; a <code>RESERVED</code> -bank read can expose a tag's access and kill passwords (see Notes)
Interaction Pattern	Publish / Subscribe (device-emitted, unsolicited; no request/response, no <code>response.code</code>)
Related	<code>alerts</code> , <code>heartBeatEVT</code> , <code>mqttConnEVT</code> , <code>set_operating_mode</code>

When to Use

Consume `dataEVT` to receive the tags an active inventory finds — it is the reader's sole tag-output stream:

- **Collect tag reads** — subscribe to the DATA endpoint's data topic and process each event's `data.tagData[]` (the always-present `EPCid` plus any enabled memory-bank and telemetry fields).
- **Collect barcode reads** — the same stream carries `data.barcodeData[]` when a barcode is scanned during the operation.
- **Read access-operation results** — when `accessOperations` are armed, each read's `accessResults[]` reports the outcome of the configured READ/WRITE/LOCK/KILL.

`dataEVT` is **device-emitted** — there is no request/response and **no polling equivalent**; you cannot fetch a tag read on demand. Configure it first with `set_operating_mode`, start the stream with `control_operation` `START`, and stop it with `STOP`. If it is already running and nothing arrives, work through [Emission](#).

Topic Mapping

`dataEVT` is device-emitted and has no `cmd` / `resp` topics. It is published on the DATA endpoint's data topic.

Message	Direction	Topic	Note
Event	Reader → App	<code><tenantId>/DATA/clients/data1event/<serial></code>	The first data channel's tag stream.
Event	Reader → App	<code><tenantId>/DATA/clients/data2event/<serial></code>	The second data channel. Currently not supported: the reader does not yet support a <code>DATA2</code> endpoint, so nothing is published here.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the

reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

The exact middle segment is the topic configured in the DATA endpoint's `publishTopics[1]` — the slot, not the topic text, is what routes tag data. See [Emission](#).

See [Topics and Channels](#).

Key Fields

Every `dataEVT` payload field below is read-only device output; the final **Emission gate** bullet is a request-side control (what arms the stream), not a payload field.

- `type` (event) — the active operating-mode profile echoed on the read (e.g. `BALANCED_PERFORMANCE`); the reader emits the same profile on every event of a run.
- `timestamp` (event) — ISO-8601 time the event was generated; the only required top-level field.
- `data.tagData[]` (event) — the batch of RFID reads. Each read always carries `EPCid` (hex tag identifier) plus the required `eventNum` (per-record sequence number).
- `EPC / TID / USER / RESERVED` (event) — memory-bank contents as hex, each present only when its bank is enabled; a `RESERVED` read can expose the tag's access/kill passwords (see Notes).
- `peakRssi / phase / channel / seenCount` (event) — per-read telemetry: RSSI in dBm, phase in degrees, the raw channel value, and the seen-count — each gated by its `tagMetaDataToEnable` flag.
- `accessResults[]` (event) — one string per armed access operation (READ/WRITE/LOCK/KILL); present only when `accessOperations` are configured.
- `data.barcodeData[]` (event) — barcode reads (`decodedBarcode` , `symbology`) when a barcode is scanned during the operation.
- **Emission gate** (request) — the stream flows only when an endpoint, an operating mode and a running inventory all line up. No active inventory, no `dataEVT` . The full chain, with a check at each link, is [Emission](#).

Emission

Every command in this chain can return `code: 0` while the stream stays silent. The response-code catalogue is a closed 39-member set and **not one member describes a data endpoint that is missing, inactive or wrongly addressed** — so a silent stream produces no error to read. Silence is the signal, and this section is the checklist.

Work down it in order. Each link has something you can verify.

1. Subscribe Before You Start

Tag data is published with `retain: false` on all three telemetry channels, so **anything published before your subscription is live is gone and is never redelivered**. A client that sends `START` and *then* subscribes sees an empty stream that is not a configuration fault at all.

2. A DATA Endpoint Must Exist

`epType` is a closed seven-member enum — `MGMT`, `MGMT_EVT`, `CTRL`, `DATA1`, `DATA2`, `SOTI`, `MDM` — and only the data types carry tag and barcode data. `DATA2` is currently not supported, so create a `DATA1` endpoint with `config_endpoint` using `operation: add`.

Verify: `get_endpoint_config` and read the returned `activeEndpoints`.

3. It Must Be Activated

`activate` is a boolean and **its default is false**, so an endpoint you did not explicitly activate is not carrying anything. Send `activate: true` for the data endpoint of type `DATA1` — on the `add`, or with an `update` if the endpoint was saved dormant.

⚠ THE TRAP THAT PRODUCES A SILENT STREAM

`publishTopics` is a **three-slot positional array, and it is the position that routes** — slot 0 is command replies, slot 1 is asynchronous events, slot 2 is the connection event and the Last Will. On a data endpoint **slot 1 carries the tag data**. The conventional names `resp` / `event` / `rfid` are a convention and are **not** auto-detected.

So a single-entry payload — one topic, in slot 0 — lands in the **reply** slot and streams nothing. The endpoint is created, the command succeeds, and no tag data ever arrives. A DATA endpoint needs **two** entries: a filler in slot 0 and the tag-data topic in slot 1.

4. An Operating Mode Must Be Configured, and It Does Not Survive a Reboot

Set the profile and the metadata flags with `set_operating_mode`. Two things to know:

- **The mode is not persistent.** A reboot discards it and the reader returns to a default the contract does not name, so re-apply the mode after any reboot rather than assuming it held.
- **Do not configure `FAST_READ`.** The contract states that data events are not currently supported under that profile, and describes the profile itself as not currently supported — pick another profile.

5. Start the Inventory

Send `control_operation` `START`. Stop with `STOP`.

❗ **CHECK WHAT THE MODE HAS ARMED BEFORE YOU SEND `START`**

`START` executes whatever `set_operating_mode` last configured, **including destructive accessOperations** — see the caution in [Notes and Caveats](#) before starting a production inventory.

Still Silent?

Two causes that look like a broken stream and are not:

- **An `INCLUDE` post filter that matches nothing.** A filter reporting only matching tags will report none if the pattern matches none. Read it back with `get_post_filter` — see [Shaping](#).
- **A reconfiguration that was refused.** Code `35` rejects a change to an *active* endpoint and advises deactivating it first. The contract does not document how to deactivate an endpoint directly, so instead of updating the active endpoint in place, do one of the following:
 - Delete the endpoint, then add it again with the new settings.
 - Activate a different endpoint first, then update the original endpoint.

One thing not to rely on: watching for an `mqttConnEVT` as proof the endpoint came up. A connection event shows a session, not that tag data is routed to slot 1, so use `get_endpoint_config` instead.

Telemetry Payload

The envelope is `type`, `timestamp` (the only required field), and `data`. Pick a **read-detail** level below; the Example and Schema update together — from a base tag batch, to reads enriched with EPC/TID and reader telemetry, to access-operation results and a barcode read. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete batch.

Read detail

Tag data event



Telemetry payload

Example: base tag data (abridged)

```
1  {
2    "type": "BALANCED_PERFORMANCE" ,
3    "timestamp": "2026-06-16T06:03:36.894Z" ,
4    "data": {
5      "tagData": [
6        {
7          "EPCid": "E28011B0A506007D3DD07C0F" ,
8          "eventNum": 41 ,
9          "peakRssi": -73 ,
10         "seenCount": 256
11       },
12       {
13         "EPCid": "311100000000000000000000" ,
14         "eventNum": 42 ,
15         "peakRssi": -50 ,
16         "seenCount": 256
17       },
18       {
19         "EPCid": "11113333880600890630DBEC" ,
20         "eventNum": 43 ,
21         "peakRssi": -53 ,
22         "seenCount": 256
23       },
24       {
25         "EPCid": "000011C1A5000062F7952926" ,
26         "eventNum": 44 ,
27         "peakRssi": -65 ,
28         "seenCount": 256
29       },
30       {
31         "EPCid": "E200001B880600870630DBEB" ,
32         "eventNum": 45 ,
33         "peakRssi": -44 ,
34         "seenCount": 256
35       },
36       {
37         "EPCid": "1111333300004001B043ACB2" ,
38         "eventNum": 46 ,
39         "peakRssi": -36 ,
40         "seenCount": 256
41       },
42       {
43         "EPCid": "11113333333344445555AAAA" ,
44         "eventNum": 47 ,
45         "peakRssi": -32 ,
46         "seenCount": 256
47       },

```

```

48      {
49          "EPCid" : "11113333A5000062F798E5B6" ,
50          "eventNum" : 48 ,
51          "peakRssi" : -34 ,
52          "seenCount" : 256
53      },
54      {
55          "EPCid" : "11113333333344445555AAAA" ,
56          "eventNum" : 49 ,
57          "peakRssi" : -36 ,
58          "seenCount" : 256
59      },
60      {
61          "EPCid" : "11115555222233334444A9B6" ,
62          "eventNum" : 50 ,
63          "peakRssi" : -43 ,
64          "seenCount" : 256
65      },
66      {
67          "EPCid" : "111133332222333344445555" ,
68          "eventNum" : 51 ,
69          "peakRssi" : -44 ,
70          "seenCount" : 256
71      },
72      {
73          "EPCid" : "111133332222333344445550" ,
74          "eventNum" : 52 ,
75          "peakRssi" : -43 ,
76          "seenCount" : 256
77      },
78      {
79          "EPCid" : "FFFFFFFFE3435363742426666" ,
80          "eventNum" : 53 ,
81          "peakRssi" : -51 ,
82          "seenCount" : 256
83      },
84      {
85          "EPCid" : "11113333333344445555AAAA" ,
86          "eventNum" : 54 ,
87          "peakRssi" : -46 ,
88          "seenCount" : 256
89      },
90      {
91          "EPCid" : "11113333880600900630DBF1" ,
92          "eventNum" : 55 ,
93          "peakRssi" : -52 ,
94          "seenCount" : 256
95      },
96      {
97          "EPCid" : "E200001B880600840630DBE2" ,
98          "eventNum" : 56 ,

```

```

99         "peakRssi" : -57 ,
100         "seenCount" : 256
101     },
102     {
103         "EPCid" : "111133332222333344445555" ,
104         "eventNum" : 57 ,
105         "peakRssi" : -63 ,
106         "seenCount" : 256
107     },
108     {
109         "EPCid" : "111133332222333344445555" ,
110         "eventNum" : 58 ,
111         "peakRssi" : -63 ,
112         "seenCount" : 256
113     },
114     {
115         "EPCid" : "111133332222333344445555" ,
116         "eventNum" : 59 ,
117         "peakRssi" : -62 ,
118         "seenCount" : 256
119     },
120     {
121         "EPCid" : "E200001B880600870630DBEB" ,
122         "eventNum" : 60 ,
123         "peakRssi" : -44 ,
124         "seenCount" : 256
125     },
126     {
127         "EPCid" : "000011C1A5000062F7952926" ,
128         "eventNum" : 61 ,
129         "peakRssi" : -66 ,
130         "seenCount" : 256
131     },
132     {
133         "EPCid" : "11113333B22FB22FB22FB22F" ,
134         "eventNum" : 62 ,
135         "peakRssi" : -65 ,
136         "seenCount" : 256
137     },
138     {
139         "EPCid" : "AAAABBBB11112222CCCCDDDD" ,
140         "eventNum" : 63 ,
141         "peakRssi" : -60 ,
142         "seenCount" : 256
143     },
144     {
145         "EPCid" : "BEDD11112222333344445555" ,
146         "eventNum" : 64 ,
147         "peakRssi" : -45 ,
148         "seenCount" : 256
149     },

```

```

150     {
151         "EPCid" : "311100000000000000000000" ,
152         "eventNum" : 65 ,
153         "peakRssi" : -51 ,
154         "seenCount" : 256
155     },
156     {
157         "EPCid" : "11113333880600890630DBEC" ,
158         "eventNum" : 66 ,
159         "peakRssi" : -53 ,
160         "seenCount" : 256
161     },
162     {
163         "EPCid" : "1111FB63AC1F3841DADA00000000CAFEDADACAFEDADACAFE00001111" ,
164         "eventNum" : 67 ,
165         "peakRssi" : -57 ,
166         "seenCount" : 256
167     },
168     {
169         "EPCid" : "E28011B0A506007D3DD07D01" ,
170         "eventNum" : 68 ,
171         "peakRssi" : -72 ,
172         "seenCount" : 256
173     },
174     {
175         "EPCid" : "E28011B0A506007D3DD07C0F" ,
176         "eventNum" : 69 ,
177         "peakRssi" : -69 ,
178         "seenCount" : 256
179     },
180     {
181         "EPCid" : "E28011B0A506007D3DD07D00" ,
182         "eventNum" : 70 ,
183         "peakRssi" : -70 ,
184         "seenCount" : 256
185     },
186     {
187         "EPCid" : "11115555222233334444A9B6" ,
188         "eventNum" : 71 ,
189         "peakRssi" : -41 ,
190         "seenCount" : 256
191     },
192     {
193         "EPCid" : "111133332222333344445550" ,
194         "eventNum" : 72 ,
195         "peakRssi" : -45 ,
196         "seenCount" : 256
197     },
198     {
199         "EPCid" : "311100000000000000000000" ,
200         "eventNum" : 73 ,

```

```
201         "peakRssi" : -65 ,
202         "seenCount" : 256
203     },
204     {
205         "EPCid" : "111133332222333344445555" ,
206         "eventNum" : 74 ,
207         "peakRssi" : -61 ,
208         "seenCount" : 256
209     },
210     {
211         "EPCid" : "A1B2222BBBB" ,
212         "eventNum" : 75 ,
213         "peakRssi" : -65 ,
214         "seenCount" : 256
215     }
216 ]
217 }
218 }
```

Telemetry payload

★ required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
1	type	enum	FAST_READ CYCLE_COUNT DENSE_READERS OPTIMAL_BATTERY BALANCED_PERFORMANCE ADVANCED	BALANCED_PERFORMANCE
2	timest amp★	string	—	2026-06-16T06:03:36.894Z
3	data	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
4	tagData	array	—	—
5	EP C * id	string	—	000011C1A5000062F7952926
6	EP C	string	—	D98A3000AAAABBBB11112222CCCC
7	TI D	string	—	e20060030178529d
8	US ER	string	—	12343333123456781234567812345678 12345678123456781234567812345678 12345678123456781234567812345678 12345678
9	RE SE RV ED	string	—	12343333123456781234567812345678 12345678123456781234567812345678 12345678123456781234567812345678 12345678
10	PC	string	—	3000
11	CR C	string	—	b06a

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
12	channel	number	—	926750
13	event [*] Number	number	—	13
14	format	string	—	epc
15	peak Rs si	number	—	-59
16	phase	number	—	0
17	seen Count	number	—	256

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
18	XP C	string	—	—
19	ac ce ss Re su lt s	array	—	["READ-EPC-Success", ...]
20	us er De fi ne d	string	—	—
21	fi rs tS ee nT im e	number	—	—
22	la st Se en Ti me	number	—	—
23	MA C	string	—	AC:3F:A4:00:2B:15
24	HO ST NA ME	string	—	RFD40-211545201D0034

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE
25	barcodeData	array	—	—
26	symbolology	string	—	0x03
27	decodedBarcode*	string	—	PWR-BGA24V78W1WW 183213949

Field Support

A tag record declares twenty fields. Two are always there, twelve are switched, and six have no switch at all.

Always present: `EPCid` and `eventNum`, the two fields the schema marks `required`. Everything else is conditional.

Switched by `tagMetadataToEnable` in `set_operating_mode`. The switch names are ALL-CAPS on purpose — they mirror the EPC Gen2 air-protocol spelling and are wire-fixed — and **four of the twelve do not match the field they enable**:

Switch	Field it enables
<code>RSSI</code>	<code>peakRssi</code>

Switch	Field it enables
PHASE	phase
CHANNEL	channel
SEENCOUNT	seenCount
EPC	EPC — or EPCid ; see below
PC , XPC , CRC , TID , USER , MAC , HOSTNAME	the identically named field

Six fields have no switch: `RESERVED` , `firstSeenTime` , `lastSeenTime` , `accessResults` , `userDefined` , `format` . The contract does not say which of them firmware can control. Two of them do arrive in practice — `RESERVED` and `accessResults` both appear when `accessOperations` are configured — so the absence of a switch is not the same as the absence of the field.

- `accessResults` arrives with configured access operations, not with metadata flags. It is populated only when you arm `accessOperations` (READ/WRITE/LOCK/KILL). A bank read enabled purely through `tagMetaDataToEnable` surfaces as the `EPC` / `TID` / `USER` metadata fields instead — which is why you can see bank contents with no `accessResults` at all. For how to read the strings, see [Telemetry Semantics](#).
- **Which field the `EPC` switch controls is not documented.** It could enable `EPC` or `EPCid` . Their descriptions differ — `EPC` is the memory bank's contents, `EPCid` the item identifier — and `EPCid` is required on every record anyway. **Code against `EPCid` and treat `EPC` as bank content you asked for.**
- **No switch declares a default**, and the contract does not say which metadata the device reports when you ask for none. What the contract's examples show is narrow and consistent: `EPCid` , `eventNum` , `peakRssi` and `seenCount` appear in **40 of 40** example records. Treat that as the observed floor, not a guarantee, and **treat every other field as optional**.

Telemetry Semantics

What each record carries. Two things determine the content of a `dataEVT` record — which fields are switched on, and which of the two reporting shapes you are receiving:

- **Which fields are present** is governed by `tagMetaDataToEnable` in `set_operating_mode` . The full switch-to-field mapping, the two always-present fields and the six with no switch at all are in [Field Support](#).
- **Two reporting shapes — and no command in this API selects between them.** A `dataEVT` arrives in one of two shapes. **Per-read:** every individual read is reported, so `channel` and

`phase` are present and `seenCount` is `1`. **Aggregated:** reads are combined and reported at intervals, so `channel` and `phase` are absent, `seenCount` counts the reads since the previous report, and `peakRssi` is the peak since that report. The reader's firmware governs which shape you get; **the contract declares no field for it and no operation carries one**, so treat the shape as something you *detect*, not something you set — **the presence of `channel` is the discriminator**.

- Despite the schema describing `channel` as a value "in MHz", the device emits a **raw channel value/index** (e.g. `9216`), **not** a ~900 MHz frequency. Treat `channel` as an opaque channel index; do not interpret it as megahertz. `channel` (and `phase`) are present only in the per-read shape.
- **Reading `accessResults` strings.** When the array is present (see [Field Support](#) for when that is), each string is one operation's outcome — `READ-EPC-Success`, or an error, or a bare `SUCCESS` for writes. The shape is one string per armed operation, in the order the operations were configured.
- **`type` echoes the profile.** `type` carries the active operating-mode profile (e.g. `BALANCED_PERFORMANCE`), and the reader emits the same value on every event of a run — so it identifies the run, not the record. One value you will not see here is `FAST_READ`; [Emission](#) explains why, and why you should not configure it. How much arrives per event is [Volume and Limits](#).

Shaping

`tagMetadataToEnable` **changes what each record carries. Post filters change which records exist.** That is the whole distinction, and it is the one worth keeping straight.

A post filter is applied after inventory, on the stream's way out, and it is the only control that changes **which tag records reach a data endpoint**. Configure it with `set_post_filter` and read it back with `get_post_filter` — the rule shape, the match methods and the `INCLUDE` / `EXCLUDE` semantics are documented there and not repeated here.

Three things to know from the `dataEVT` side, because the contract leaves them open:

- **The endpoint vocabularies differ and are not mapped.** Filters scope to `dataEpType` (`DATA_EP1` / `DATA_EP2`); endpoints are configured with `epType` (`DATA1` / `DATA2`, which is currently not supported). The contract declares both and never states the correspondence, so `DATA_EP1` to `DATA1` is the necessary reading rather than a documented one.
- **The pattern matches the *tag ID*, and the contract does not name which payload field that is.** `EPCid` is the only always-present identifier, so it is the reasonable assumption — confirm against firmware before relying on it. Nothing ties post filtering to barcode reads on the same stream.

- **Do not use `eventNum` continuity as a loss detector while a filter is installed.** Nothing in the contract says whether a suppressed read still consumes a sequence number. See [Delivery and Loss](#).

Also undocumented, and worth a read-back rather than an assumption: whether filters survive a reboot, how many rules an endpoint accepts, and what happens when two rules on one endpoint disagree.

Delivery and Loss

`dataEVT` carries no `response` or `code` — it is device-emitted, so the command response-code table does not apply and no response code is defined for any event. The only thing that tells you something went wrong is what fails to arrive.

All three telemetry channels declare `qos: 0` and `retain: false`. So loss is normal rather than exceptional, nothing is replayed to a subscriber that was not listening, and [Delivery and Retry Semantics](#) has the full treatment — including that QoS is configurable per publish topic, which changes the transport guarantee without making the reader re-send anything it already dropped.

`eventNum` is the only ordering signal on the wire. It is a per-record sequence counter, and the contract documents no wrap-around, no reset rule and no scope — so **do not assume the counter restarts between runs, or that it does not**. A break in the sequence is a prompt to investigate, not proof of loss: it cannot distinguish a dropped publish from a record the reader never emitted. And a batch arriving shortly after `STOP` is a late buffered batch, not a gap — see [Notes and Caveats](#).

Volume and Limits

Firmware caps a whole `dataEVT` message at 6144 bytes, and there is no record-count limit. The current contract states neither limit; the byte cap is a firmware limit that the device owners confirmed, and firmware notes it can be increased to support batching, so treat it as a current-build number. An over-size message is **dropped by the reader, with no error, and the connection stays up**: the symptom is a message that never arrives. Size a buffer against 6144 bytes and a variable record count — not against a tag count. The serialisation is not stated: any tags-per-event arithmetic assumes minified JSON.

The declared controls live on the endpoint as `batching`: `reportingInterval` (1000–60000 ms) and `maxPayloadSizePerReport` (1024–5120 bytes); a `0` in either falls back to the minimum. A report is flushed on **whichever comes first, and the contract does not document which wins** when both thresholds are near. Send both members together — whether setting one and omitting the other leaves the other at its default is undocumented. The contract's examples use `1000` / `1024`.

Examples

Every contract example is in the **Read detail** dropdown under [Telemetry Payload](#), and which fields appear reflects the operating-mode configuration:

- **Tag data event** — the default fields only: `EPCid`, `eventNum`, `peakRssi` and `seenCount`.
- **Tag data with additional details** — EPC and TID memory plus `channel`, `phase`, `MAC` and `HOSTNAME`, with their `tagMetaDataToEnable` flags on. In this per-read shape `channel` and `phase` are present and `seenCount` is `1`.
- **Tag events for access operations** — a configured `READ` of the EPC, TID and RESERVED banks, with the outcomes in `accessResults`.
- **Barcode data event** — a scanned barcode, reported in `data.barcodeData[]` instead of `tagData`. Barcode reads were not exercised live; this is the contract's example.

Notes and Caveats



RESERVED BANK READS EXPOSE TAG PASSWORDS

The `RESERVED` memory bank holds a tag's **access and kill passwords**. If you configure a `READ` of the RESERVED bank, those passwords appear in `dataEVT` as the `RESERVED` hex field — treat that data as sensitive. (`EPCid` and the other bank contents are physical-tag data, not device or broker credentials.)

- **STOP is not instantaneous for buffered reads.** `control_operation` `STOP` halts the radio, but a buffered in-flight `dataEVT` batch may still be delivered a few seconds after `STOP` ($\approx 3\text{--}5$ s observed on RFD40). Do not treat a `dataEVT` arriving shortly after `STOP` as an error, and do not stop listening the instant you send `STOP`.

Start the stream with `control_operation` `START` and stop it with `STOP`. Because `dataEVT` reflects the operating mode, **any armed destructive `accessOperations` (WRITE/LOCK/KILL) execute on the tags read during the run** — confirm the mode is read-only before starting (see [set_operating_mode § Notes](#)). For the concept-level view, see [The Tag Data Stream, Choose Shared or Dedicated Data Endpoints](#), and `dataEVT`.

Related:  [control_operation](#) ·  [set_operating_mode](#) ·  [config_endpoint](#) ·  [What to Monitor and Why](#) ·  [The Tag Data Stream](#) ·  [Choose Shared or Dedicated Data Endpoints](#)